Classification by Decision Tree Induction

ID3, C4.5 and CART

Dr. Nikhilanand Arya

IIITA

February 24, 2026

Introduction to Decision Trees

Decision Tree Induction

- Supervised learning algorithm
- Used for classification and regression
- Tree is constructed in a top-down, recursive, divide-and-conquer manner
- At start, all the training examples are at the root
- Examples are partitioned recursively based on selected attributes
- At each node, attributes are selected using a heuristic/statistical measure such as information gain or Gini index

Tree Structure

- Root Node $\rightarrow$ Internal Node $\rightarrow$ Branch $\rightarrow$ Leaf Node

Pros and Cons

Pros

- Easy to explain (even for non-expert)
- Easy to implement (widely available in software)
- Efficient in training and prediction
- Can tolerate missing data
- White-box model (interpretable)
- No need to normalize data
- Non-parametric: no assumption on data distribution and no assumption on attribute independence
- Works on various attribute types (categorical and continuous)

Pros and Cons

Cons

- Unstable – sensitive to noise
- Accuracy may not be optimal (depends on data)
- Finding globally optimal tree is NP-hard (greedy heuristics are used)
- Overfitting problem

ID3 Algorithm

Overview

- Developed by Ross Quinlan (1986)
- Based on **Information Gain**
- Uses **Entropy** as impurity measure
- Greedy top-down recursive approach
- Works only with categorical attributes
- No pruning; therefore susceptible to overfitting

Core Idea

Select the attribute that gives the largest reduction in uncertainty at each node.

Splitting Measures: Information Gain

Entropy

- Entropy is an information-theoretic measure of uncertainty.
- For discrete random variable Y with values $\{y_1, \ldots, y_m\}$:

$$H(Y) = -\sum_{i=1}^{m} p_i \log_2(p_i), \qquad p_i = P(Y = y_i)$$

- Higher entropy $\rightarrow$ higher uncertainty
- Lower entropy $\rightarrow$ lower uncertainty

Conditional Entropy

$$H(Y \mid X) = \sum_{x} P(x) H(Y \mid X = x)$$

Information Gain: An Attribute Selection Measure

Attribute Selection

- Select the attribute with the **highest Information Gain** (used by ID3 and C4.5)
- Let p_i be the probability that a tuple in dataset D belongs to class C_i:

$$p_i = \frac{|C_i, D|}{|D|}$$

Expected Information

$$Info(D) = -\sum_{i=1}^{m} p_i \log_2(p_i)$$

Information After Splitting on Attribute A

Weighted Entropy

Suppose attribute A splits dataset D into v partitions $D_1, D_2, \ldots, D_v$.

$$Info_A(D) = \sum_{j=1}^{v} \frac{|D_j|}{|D|} \times Info(D_j)$$

Interpretation

The post-split information is a weighted average of the entropy of the resulting partitions.

Information Gain Formula

Information Gain

$$Gain(A) = Info(D) - Info_A(D)$$

Selection Rule

Choose the attribute with maximum Gain.

Intuition

- Higher Gain $\rightarrow$ better purity improvement
- Lower Gain $\rightarrow$ poorer split

Steps of ID3 Algorithm

1. Compute entropy of dataset S
2. For each attribute A, compute $Gain(S, A)$
3. Select the attribute with highest Gain
4. Split the dataset
5. Repeat recursively until:
   - all samples belong to the same class, or
   - no attributes are left

Play Tennis Dataset

Outlook	Temp	Humidity	Wind	Play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Step 1: Calculate Entropy of Dataset

Dataset Statistics

Total samples $= 14$, Yes $= 9$, No $= 5$.

$$\begin{aligned} \textit{Entropy}(S) &= -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} \\ &= -(0.643)\log_2(0.643) - (0.357)\log_2(0.357) \end{aligned}$$

Result

$\textit{Entropy}(S) = 0.940$ — impurity before any split.

Step 2: Weighted Entropy Calculation (All Attributes)

Outlook

Sunny $(5 : 2Y, 3N) \rightarrow 0.971$; Overcast $(4 : 4Y, 0N) \rightarrow 0$; Rain $(5 : 3Y, 2N) \rightarrow 0.971$.

$$WeightedEntropy(Outlook) = \frac{5}{14}(0.971) + \frac{4}{14}(0) + \frac{5}{14}(0.971) = 0.694$$

$$Gain(Outlook) = 0.940 - 0.694 = 0.246$$

Weighted Entropy (Remaining Attributes)

Temperature

Hot $(4 : 2Y, 2N) \rightarrow 1.0$; Mild $(6 : 4Y, 2N) \rightarrow 0.918$; Cool $(4 : 3Y, 1N) \rightarrow 0.811$.

$$WeightedEntropy(Temp) = \frac{4}{14}(1.0) + \frac{6}{14}(0.918) + \frac{4}{14}(0.811) = 0.911$$

$$Gain = 0.940 - 0.911 = 0.029$$

Humidity

High $(7 : 3Y, 4N) \rightarrow 0.985$; Normal $(7 : 6Y, 1N) \rightarrow 0.592$.

$$WeightedEntropy(Humidity) = 0.789, \quad Gain = 0.151$$

Wind

Weak $(8 : 6Y, 2N) \rightarrow 0.811$; Strong $(6 : 3Y, 3N) \rightarrow 1.0$.

$$WeightedEntropy(Wind) = 0.892, \quad Gain = 0.048$$

Step 2: Information Gain for All Attributes

Comparison

Attribute	Weighted Entropy	Gain
Outlook	0.694	0.246
Temperature	0.911	0.029
Humidity	0.789	0.151
Wind	0.892	0.048

Decision

Highest Gain $= 0.246$ (Outlook) $\Rightarrow$ Root Node $=$ Outlook.

Step 3: Split on Outlook

- Sunny $\rightarrow$ (2 *Yes*, 3 *No*) $\rightarrow$ Entropy $= 0.971$
- Overcast $\rightarrow$ (4 *Yes*, 0 *No*) $\rightarrow$ Entropy $= 0$
- Rain $\rightarrow$ (3 *Yes*, 2 *No*) $\rightarrow$ Entropy $= 0.971$

Observation

The Overcast branch is pure, so it becomes a leaf node: **Yes**.

Step 4: Process Sunny Subset

Sunny subset: 5 samples; Yes = 2, No = 3; Entropy = 0.971.

Gain for Remaining Attributes

Attribute	Gain (Sunny subset)
Temperature	0.020
Humidity	0.971
Wind	0.020

Selection

Highest Gain = Humidity.

Step 4: Process Sunny Subset (Complete Numerical)

Sunny subset $= 5$ samples; Yes $= 2$, No $= 3$.

$$Entropy(Sunny) = -\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5}$$

$$= -(0.4)\log_2(0.4) - (0.6)\log_2(0.6) = 0.971$$

Interpretation

The Sunny subset remains impure and requires another split.

Sunny Subset: Gain for Temperature

Sunny subset records:

- Hot $\rightarrow (0Y, 2N) \rightarrow Entropy = 0$
- Mild $\rightarrow (1Y, 1N) \rightarrow Entropy = 1$
- Cool $\rightarrow (1Y, 0N) \rightarrow Entropy = 0$

$$WeightedEntropy(Temp \mid Sunny) = \frac{2}{5}(0) + \frac{2}{5}(1) + \frac{1}{5}(0) = 0.4$$

$$Gain = 0.971 - 0.4 = 0.571$$

Sunny Subset: Gain for Humidity

- High $\rightarrow (0Y, 3N) \rightarrow Entropy = 0$
- Normal $\rightarrow (2Y, 0N) \rightarrow Entropy = 0$

$$WeightedEntropy(Humidity \mid Sunny) = \frac{3}{5}(0) + \frac{2}{5}(0) = 0$$

Result

$Gain = 0.971 - 0 = 0.971$; Maximum Gain = Humidity.

Sunny Subset: Gain for Wind

- Weak $\rightarrow (1Y, 2N)$; Entropy $= 0.918$
- Strong $\rightarrow (1Y, 1N)$; Entropy $= 1$

$$WeightedEntropy(Wind \mid Sunny) = \frac{3}{5}(0.918) + \frac{2}{5}(1) = 0.951$$

$$Gain = 0.971 - 0.951 = 0.020$$

Sunny Subset: Attribute Selection

Attribute	Gain (Sunny Subset)
Temperature	0.571
Humidity	0.971
Wind	0.020

Decision

Highest Gain = Humidity; Sunny branch splits on Humidity.

Sunny Branch Split

Humidity Split

- Humidity = High $\rightarrow (0Y, 3N) \rightarrow$ **No**
- Humidity = Normal $\rightarrow (2Y, 0N) \rightarrow$ **Yes**

Observation

Sunny branch becomes pure after splitting on Humidity.

Step 5: Process Rain Subset

Rain subset: 5 samples; Yes = 3, No = 2; Entropy = 0.971.

Gain Calculations

Attribute	Gain (Rain subset)
Temperature	0.020
Humidity	0.020
Wind	0.971

Selection

Highest Gain = Wind.

Rain Branch Split

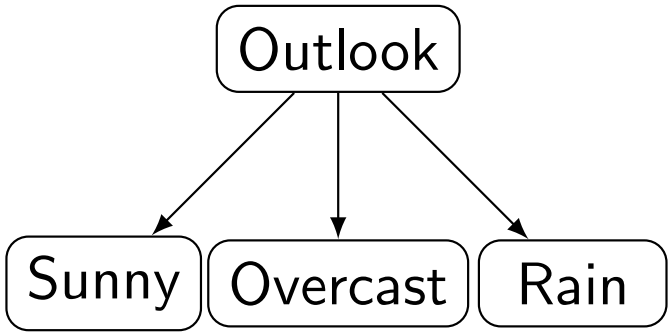
Wind Split

- Wind = Weak $\rightarrow (3Y, 0N) \rightarrow$ **Yes**
- Wind = Strong $\rightarrow (0Y, 2N) \rightarrow$ **No**

Observation

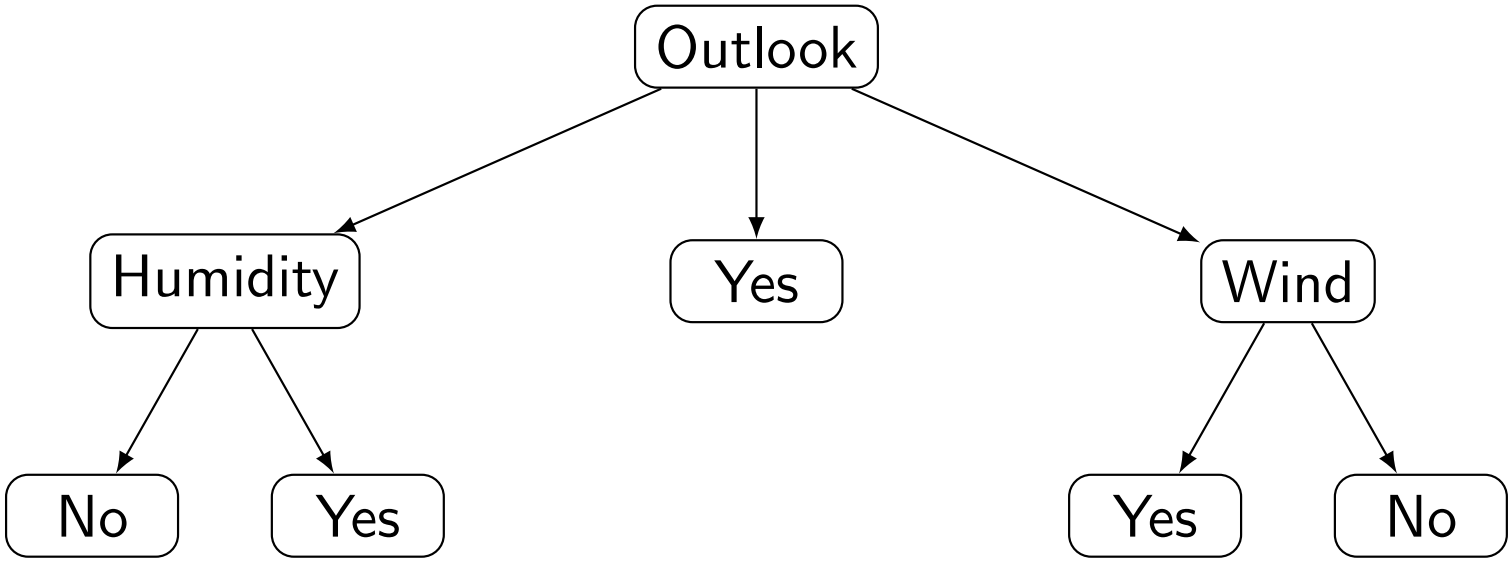
Rain branch becomes pure after splitting on Wind.

Tree Construction – Level 2



Overcast branch is already a Yes leaf; Sunny and Rain are processed further.

Final Decision Tree (ID3)



Branch order: Sunny $\rightarrow$ Humidity; Overcast $\rightarrow$ Yes; Rain $\rightarrow$ Wind.

C4.5 Algorithm

Overview

- Developed by Ross Quinlan as successor to ID3
- Uses **Gain Ratio** instead of Information Gain
- Handles continuous attributes
- Handles missing values
- Uses pruning to reduce overfitting

Steps of C4.5 Algorithm

1. Compute Gain Ratio for all attributes
2. Select attribute with highest Gain Ratio
3. Handle continuous attributes by finding the best threshold
4. Split dataset
5. Repeat recursively
6. Apply post-pruning

Problem with Information Gain

Bias

Information Gain favors attributes with many distinct values.

Example: Student ID

- Student ID has a unique value for every record
- A split on ID can make each partition pure
- Entropy becomes 0 and Gain becomes very high
- The resulting split is misleading and biased

Solution

Use **Gain Ratio** to normalize Information Gain by the intrinsic information of the split.

Gain Ratio Formula

Step 1: Information Gain

$$Gain(A) = Info(D) - Info_A(D)$$

Step 2: Split Information

$$SplitInfo(A) = -\sum_{j=1}^{v} \frac{|D_j|}{|D|} \log_2 \left(\frac{|D_j|}{|D|} \right)$$

Step 3: Gain Ratio

$$GainRatio(A) = \frac{Gain(A)}{SplitInfo(A)}$$

C4.5 Numerical: SplitInfo for Outlook

From the Play Tennis dataset, Outlook splits into Sunny (5), Overcast (4), Rain (5).

$$SplitInfo(Outlook) = -\left[\frac{5}{14}\log_2\frac{5}{14} + \frac{4}{14}\log_2\frac{4}{14} + \frac{5}{14}\log_2\frac{5}{14}\right] = 1.577$$

$$Gain(Outlook) = 0.246$$

$$GainRatio = \frac{0.246}{1.577} = 0.156$$

Gain Ratio Comparison

Attribute	Gain	SplitInfo	Gain Ratio
Outlook	0.246	1.577	0.156
Temperature	0.029	1.556	0.019
Humidity	0.151	1.000	0.151
Wind	0.048	0.985	0.049

Decision

Highest Gain Ratio = Outlook $\Rightarrow$ Root Node = Outlook.

How to Handle Continuous-Valued Attributes?

Method 1: Discretization

Convert continuous values into categorical intervals, e.g. Age $= \{< 20, 20\text{–}30, 30\text{–}40, 40\text{–}50, > 50\}$.

Method 2: Find the Best Split Point

- Sort values: 15, 18, 21, 22, 24, 25, 29, 31, ...
- Candidate split point: $P = (a_i + a_{i+1})/2$
- Example candidates: 16.5, 19.5, 21.5, 23, 24.5, 27, 30, ...
- Select the point giving maximum Information Gain
- Final binary split: $A \leq P$ vs. $A > P$

Handling Continuous Attributes

C4.5 Procedure

- Sort attribute values
- Try threshold splits
- Example: $Temp \leq 75^{\circ}$
- Choose the threshold with the highest Gain Ratio

Post-Pruning in C4.5

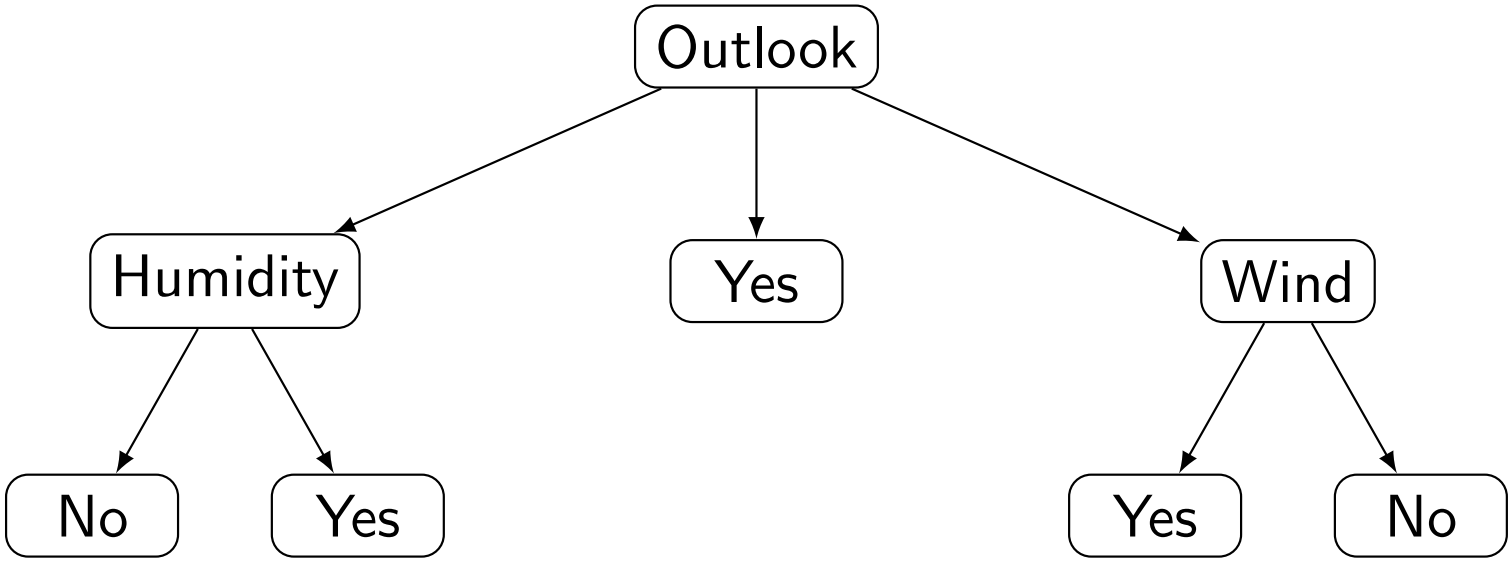
Why Prune?

ID3 builds a full tree and is therefore vulnerable to overfitting.

Error-Based Pruning

- C4.5 estimates the error associated with subtrees
- Replace a subtree with a leaf if the estimated error does not improve
- The result is a smaller, more general tree

Final Decision Tree (C4.5)



The resulting structure is compact and uses the same Play Tennis logic shown in the source material.

CART Algorithm

Overview

- Developed by Breiman et al.
- CART = Classification and Regression Trees
- Uses **Gini Index**, not Entropy
- Produces binary splits only
- Supports classification and regression
- Uses cost-complexity pruning

Gini Index

Definition

Gini measures impurity of a dataset:

$$Gini(S) = 1 - \sum_{i=1}^{m} p_i^2$$

Binary Classification

$$Gini(S) = 1 - \left(p^2 + (1-p)^2\right)$$

- Gini $= 0 \rightarrow$ pure node
- Higher Gini $\rightarrow$ more impurity

Step 1: Gini of Dataset

Total $= 14$; Yes $= 9$; No $= 5$.

$$Gini(S) = 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2$$

$$= 1 - (0.413 + 0.127) = 1 - 0.540 = 0.459$$

Result

$Gini(S) = 0.459$.

CART: Split Outlook (Sunny vs Others)

Sunny

5 records: 2Y, 3N

$$Gini(Sunny) = 1 - \left(\frac{2}{5}\right)^2 - \left(\frac{3}{5}\right)^2 = 0.48$$

Others

9 records: 7Y, 2N

$$Gini(Others) = 1 - \left(\frac{7}{9}\right)^2 - \left(\frac{2}{9}\right)^2 = 0.346$$

Weighted Gini

$$\frac{5}{14}(0.48) + \frac{9}{14}(0.346) = 0.394$$

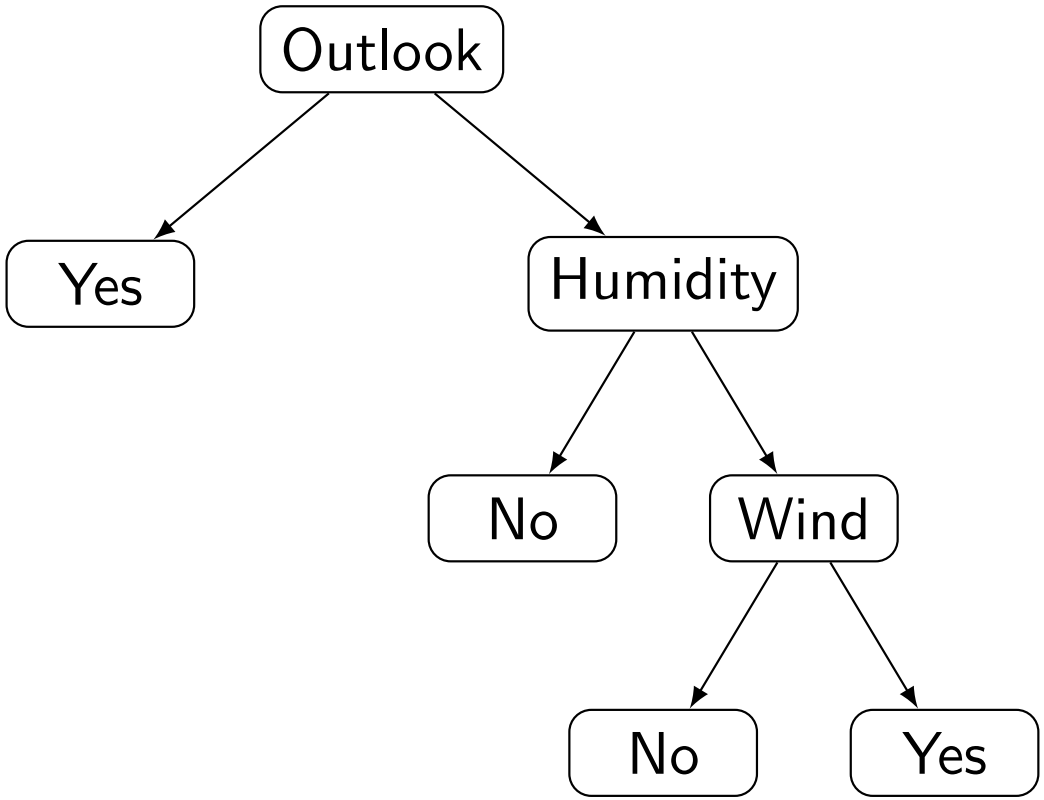
Binary Split Comparison (Outlook)

Binary Split	Weighted Gini
Sunny vs Others	0.394
Overcast vs Others	0.357
Rain vs Others	0.394

Decision

Lowest Gini = 0.357. Root split: Outlook = Overcast?

Final CART Tree (Binary Structure)



Binary structure: every internal node creates exactly two branches.

Splitting Criteria Comparison

Feature	ID3	C4.5	CART
Split Measure	Information Gain	Gain Ratio	Gini Index
Split Type	Multi-way	Multi-way	Binary only
Handles Continuous	No	Yes	Yes
Pruning	No	Yes	Yes
Regression	No	No	Yes

Formula Comparison

ID3

$$Gain(A) = H(D) - H_A(D)$$

C4.5

$$GainRatio(A) = \frac{Gain(A)}{SplitInfo(A)}$$

CART

$$Gini(S) = 1 - \sum_i p_i^2$$

Numerical Comparison (Play Tennis)

Algorithm	Root Attribute	Measure	Value
ID3	Outlook	Gain	0.246
C4.5	Outlook	GainRatio	0.156
CART	Outlook	Gini	0.357

Observation

All three select Outlook as the root, but their splitting logic differs.

Decision Tree Implementation in Python

Implementation Roadmap

1. Load the Play Tennis dataset
2. Encode categorical attributes
3. Split data into training and testing sets
4. Train a Decision Tree classifier
5. Inspect the learned tree and splitting criterion
6. Evaluate predictions using standard metrics

Python Library

scikit-learn provides a practical implementation of CART-style decision trees through `DecisionTreeClassifier`.

Step 1: Import Required Libraries

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
```

Purpose

These libraries provide data handling, model training, visualization, and evaluation utilities.

Step 2: Create the Play Tennis Dataset

```
data = {
    'Outlook': ['Sunny','Sunny','Overcast','Rain','Rain','Rain',
                'Overcast','Sunny','Sunny','Rain','Sunny','Overcast
                    ',
                'Overcast','Rain'],
    'Temperature': ['Hot','Hot','Hot','Mild','Cool','Cool','Cool',
                    'Mild','Cool','Mild','Mild','Mild','Hot','Mild'
                        ],
    'Humidity': ['High','High','High','High','Normal','Normal','
        Normal',
                 'High','Normal','Normal','Normal','High','Normal',
                     'High'],
    'Wind': ['Weak','Strong','Weak','Weak','Weak','Strong','Strong'
        ,
                 'Weak','Weak','Weak','Strong','Strong','Weak','Strong'
                     ],
    'Play': ['No','No','Yes','Yes','Yes','No','Yes',
                 'No','Yes','Yes','Yes','Yes','Yes','No']
}

df = pd.DataFrame(data)
print(df)
```

Step 3: Encode Categorical Features

Why Encoding?

Most scikit-learn tree estimators require numerical input. Therefore, categorical values such as `Sunny`, `Hot`, and `Weak` must be converted into numbers.

Label Encoding

Simple mapping, e.g.

- Sunny $\rightarrow$ 2
- Overcast $\rightarrow$ 0
- Rain $\rightarrow$ 1

Useful for compact classroom examples.

One-Hot Encoding

Creates binary indicator columns.

- Outlook_Sunny
- Outlook_Overcast
- Outlook_Rain

Avoids imposing an artificial ordering on categories.

Step 4: Encode the Dataset with pandas

```
from sklearn.preprocessing import LabelEncoder

encoded = df.copy()

for column in encoded.columns:
    encoder = LabelEncoder()
    encoded[column] = encoder.fit_transform(encoded[column])

X = encoded.drop('Play', axis=1)
y = encoded['Play']
```

Important

The same encoding learned from the training data should be applied consistently to future/test observations.

Step 5: Train–Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

print(X_train.shape)
print(X_test.shape)
```

Interpretation

- Training set: used to learn the tree.
- Test set: held out for evaluating generalization.
- `random_state`: makes the split reproducible.
- `stratify=y`: approximately preserves class proportions.

Step 6: Build and Train the Decision Tree

```
model = DecisionTreeClassifier(
    criterion='gini',
    max_depth=3,
    random_state=42
)

model.fit(X_train, y_train)
```

criterion

'gini' uses the Gini impurity, as in CART.

max_depth

Limits tree depth and helps control overfitting.

Python Parameters and Their Meaning

Parameter	Meaning
`criterion='gini'`	Selects Gini impurity for measuring split quality.
`criterion='entropy'`	Uses Shannon entropy for split evaluation.
`criterion='log_loss'`	Uses log-loss based splitting in supported versions.
`max_depth`	Maximum depth of the learned tree.
`min_samples_split`	Minimum samples required to split an internal node.
`min_samples_leaf`	Minimum samples required in a leaf node.
`random_state`	Controls reproducibility where randomness is involved.

Step 7: Make Predictions

```
y_pred = model.predict(X_test)

print('Predicted:', y_pred)
print('Actual:   ', y_test.to_numpy())

# Class probabilities
probabilities = model.predict_proba(X_test)
print(probabilities)
```

Prediction Output

`predict()` returns the most likely class, while `predict_proba()` returns the estimated probability distribution over classes.

Step 8: Evaluate the Model

```
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
report = classification_report(y_test, y_pred)

print('Accuracy:', accuracy)
print('Confusion Matrix:\\n', cm)
print(report)
```

Accuracy

Fraction of test samples classified correctly.

Classification Report

Includes precision, recall, F1-score, and support for each class.

Step 9: Visualize the Learned Tree

```
plt.figure(figsize=(12, 7))

plot_tree(
    model,
    feature_names=X.columns,
    class_names=['No', 'Yes'],
    filled=True,
    rounded=True
)

plt.title('Decision Tree Classifier')
plt.show()
```

Interpretation

Each internal node represents a learned decision rule; each branch corresponds to an outcome of that rule; each leaf gives the final prediction.

Step 10: Inspect the Learned Splits

```
from sklearn.tree import export_text

rules = export_text(
    model,
    feature_names=list(X.columns)
)

print(rules)
```

What We Obtain

- The feature selected at each node.
- The numerical threshold used by the tree.
- The branch taken when a condition is true or false.
- The predicted class at terminal nodes.

From CART Mathematics to Python Code

CART Theory

$$Gini(S) = 1 - \sum_i p_i^2$$

Choose the split with the lowest weighted impurity.

Python

```
DecisionTreeClassifier(
    criterion='gini'
)
```

The library performs the split search and tree construction automatically.

Key Connection

The mathematical criterion studied in CART is directly reflected in the model's `criterion` parameter.

Avoiding Overfitting in Python

Why Overfitting Occurs

A deep tree can keep splitting until it memorizes the training data, producing excellent training accuracy but poor test performance.

Common Controls

- `max_depth`: limit tree depth
- `min_samples_split`: require sufficient samples before splitting
- `min_samples_leaf`: prevent very small leaves
- `max_leaf_nodes`: restrict the number of leaves
- `ccp_alpha`: cost-complexity pruning parameter

Cost-Complexity Pruning in Python

```
pruned_model = DecisionTreeClassifier(
    criterion='gini',
    ccp_alpha=0.01,
    random_state=42
)

pruned_model.fit(X_train, y_train)
```

Idea

A positive ccp_alpha penalizes tree complexity. Increasing it generally produces a smaller tree, which can improve generalization.

Complete Python Implementation

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)

model = DecisionTreeClassifier(
    criterion='gini', max_depth=3, random_state=42
)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, y_pred))
```

Workflow

Data → Encode → Split → Train → Predict → Evaluate → Visualize

Practical Checklist for Decision Tree Projects

1. Understand the target variable and feature types.
2. Clean missing or invalid values appropriately.
3. Encode categorical variables consistently.
4. Separate training and test data before model fitting.
5. Select a splitting criterion: Gini, entropy, or supported alternative.
6. Control tree complexity to reduce overfitting.
7. Evaluate on unseen data using more than accuracy when appropriate.
8. Visualize and inspect the learned tree for interpretability.
9. Tune hyperparameters using cross-validation when sufficient data are available.

Key Takeaways: Python Implementation

Conceptual Link

- ID3: Information Gain + Entropy
- C4.5: Gain Ratio + pruning/continuous attributes
- CART: Gini Index + binary splitting + pruning

Implementation Insight

Python libraries automate the computational details, but understanding the splitting criteria is essential for interpreting the learned model and choosing appropriate hyperparameters.

Remember

A decision tree should not only fit the training data; it should generalize well to unseen examples.

Thank You!

Any Questions?